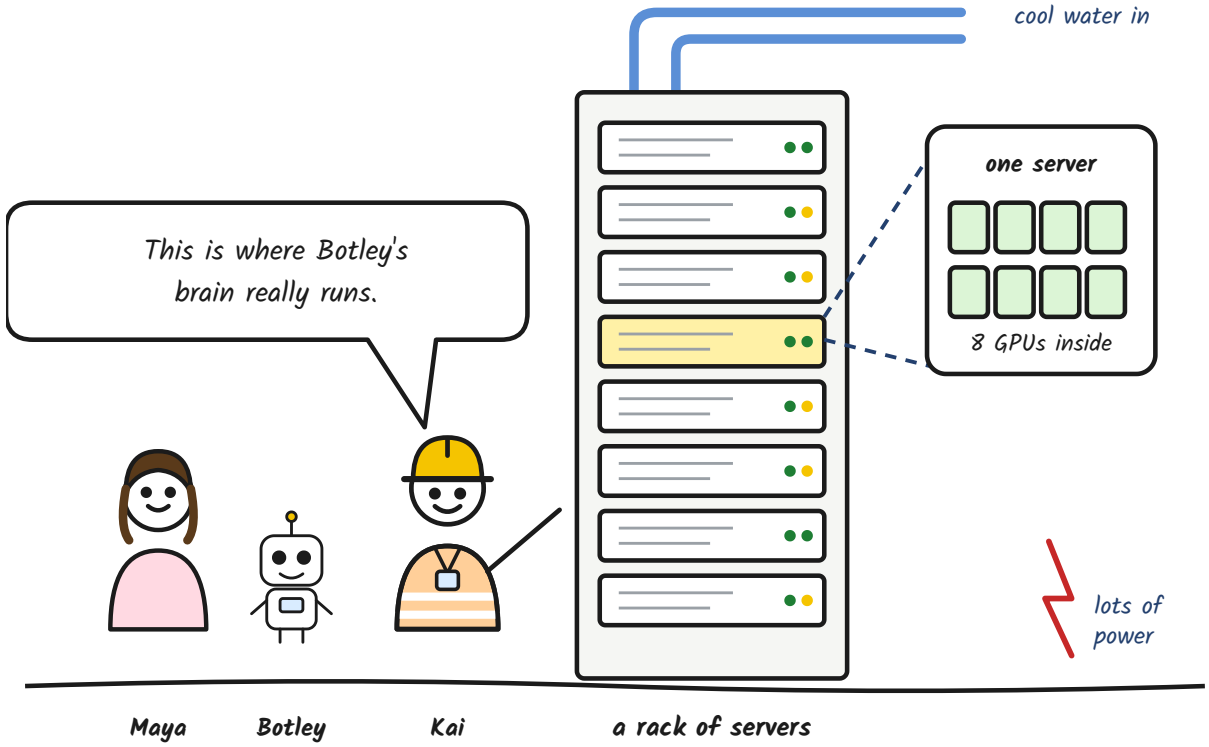
Notebook #3 from Ship It with Nimesha

Inside the Machine

No hardware background needed

How GPUs run your agent. From a single on/off switch to the data center.



Kai, an infrastructure engineer, takes Maya on a tour. Start with how any computer works, see why AI needs GPUs and why memory speed sets the pace, then walk through a real data center. Lots of drawings, no maths background needed.

How does a GPU actually work?

Why memory speed sets how fast answers flow

A walk through a real data center

Nimesha Jinarajadasa

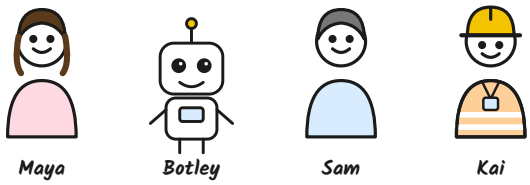
First edition, October 2026

How to use this notebook

Lots of pictures, everyday comparisons and no maths background.

This notebook explains the hardware that runs AI agents, starting from zero. You don't need to know any electronics or maths. If you have ever wondered what is really inside a GPU, why some GPUs are so much faster than others, or why GPUs are so hard to get, this is for you.

You will meet a new character. **Kai** is an infrastructure engineer who looks after the data center where Botley's model runs, and takes Maya on a tour.



Maya builds agents. **Botley** is the agent she built. **Sam** is her boss. **Kai** keeps the machines powered, cooled and connected.

One picture for the whole notebook

Think of a computer as a kitchen.

In the kitchen	In the computer
A few expert chefs	A CPU: a few clever cores.
Thousands of line cooks	A GPU: thousands of simple cores.
The counter, the fridge, the warehouse	GPU memory, SSD storage, network storage.
The conveyor belt to the cooks	Memory bandwidth: how fast numbers reach the cores.
A giant shared kitchen	A data center, with its own power and cooling.

Boxes you will see

Box	What it is for
There are no silly questions	Questions people often want to ask but don't.
Think about it	Stop and guess the answer before you read on. It helps you remember.
Careful!	Mistakes that real teams make.
Your turn	A short exercise. The answers are at the back.
Face-off	Two people or ideas argue with each other.
Geek corner	A little bit of code or math. You can skip it.
Key takeaways	The main points of the chapter.

About the numbers

Every GPU figure in this notebook was checked against NVIDIA's, AMD's, Google's and AWS's own documents in October 2026, and each chapter shows the date its facts were checked. New GPUs come out often, so check the latest documents before you decide.

What's inside

#	Chapter	The question it answers	Page
1	What happens when Botley thinks?	Where does an agent's request actually go, and what happens there?	3
2	Switches all the way down	How does a computer store a number, and what do BF16 and FP8 really mean?	6
3	The CPU: a few very fast chefs	What does a CPU do, and what is it good at?	9
4	The counter, the fridge and the warehouse	Where are the model's weights kept, and how do they reach the GPU?	12
5	The maths inside a model, gently	What work does a model actually do to write one token?	15
6	The GPU: thousands of line cooks	How is a GPU different from a CPU, and what are cores, SMs and Tensor Cores?	18
7	The real bottleneck	Why does memory bandwidth matter so much for running a model?	21
8	When one GPU isn't enough	How do several GPUs work together on one model?	25
9	Inside a data center	What does it take to keep thousands of GPUs running?	28
10	Not only NVIDIA	What other AI chips exist, and when do they matter?	31
11	How to read a GPU spec sheet	Which numbers on a spec sheet matter for running an agent's model?	34
12	What this means for your agent	How do I use all this to make my agent faster and cheaper?	37
	Back of the book	Word list and exercise answers.	40

Chapter one

1 What happens when Botley thinks?

One request's trip from a phone to a GPU and back. The rest of this notebook zooms into each stop.

Facts in this chapter last checked on 26 October 2026.



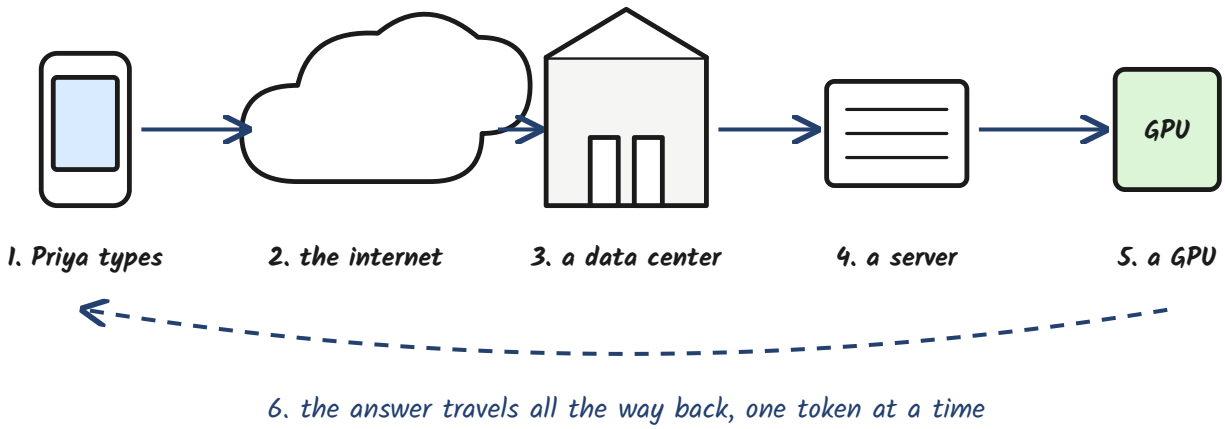
Hi Maya, I'm Kai. Come and see where Botley's brain really runs.

Maya has read the spec sheets. She knows an H100 has 80 GB of memory and that an L4 is cheaper. But numbers like "3.35 TB/s" still mean nothing to her. Why does that number matter so much? What is actually inside the box?

Kai looks after the data center where Maya's GPUs run. Kai is an infrastructure engineer: the person who keeps the machines powered, cooled, connected and healthy. Kai offers Maya a tour.

The trip of one request

Priya, one of Botley's users, types: "Find me a flight to Lisbon on Tuesday." Here is where that message goes.



1. **Priya's phone** sends the message over the internet.
2. **The internet** carries it to the right building. If Priya is on the other side of the world, the trip there and back alone takes a noticeable part of a second.
3. **A data center** receives it. Inside the building, messages move between machines in well under a millisecond.
4. **A server**, one of thousands of computers in the building, runs Botley's code. Botley decides it needs the model.
5. **A GPU** in another server reads the request and writes the answer, one token at a time.
6. **The answer** travels all the way back to Priya's phone.

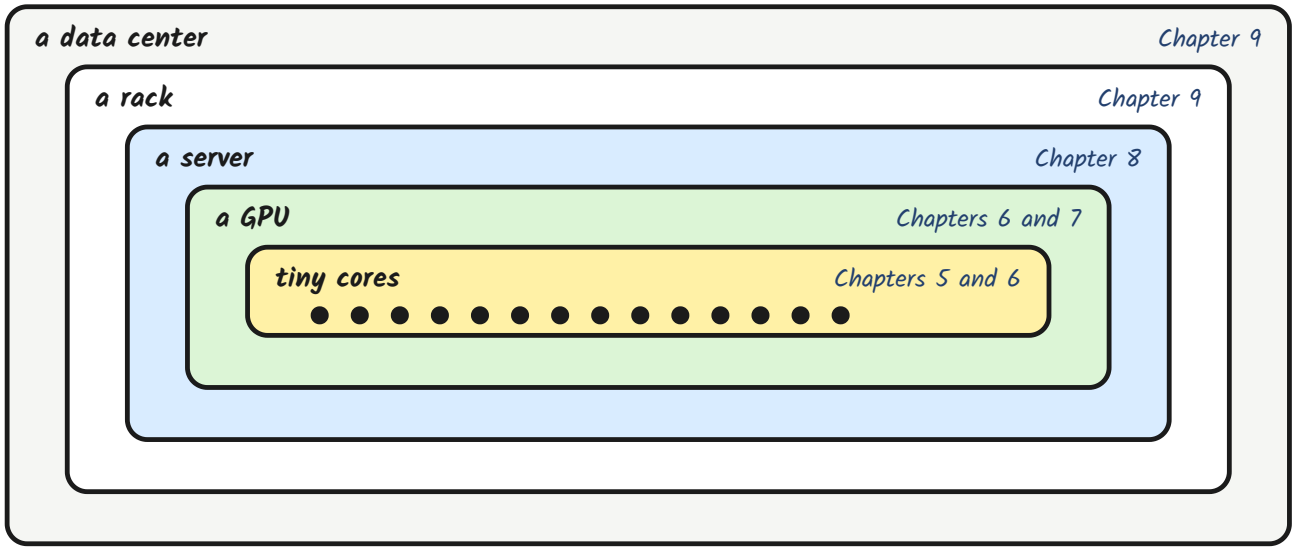
Rough timings from Jeff Dean's well-known list of "numbers everyone should know": a round trip inside one data center takes about 0.5 milliseconds, and a message from California to the Netherlands and back takes about 150 milliseconds. The list is from 2009, so treat the numbers as rough sizes, not exact values.

Think about it

Botley often calls the model ten or more times for one task, and calls tools in between. If each step needs a trip across the world and back, what happens to the total waiting time?

Zooming in

Each stop in that trip is a box inside a bigger box. A data center holds rows of racks. A rack holds servers. A server holds GPUs. A GPU holds thousands of tiny workers.



This notebook starts with the smallest box and works its way out.

We will start at the very bottom, with a single on/off switch, and build up from there:

Part	What you will learn	Chapters
How any computer works	Switches, numbers, CPUs, memory and storage	2 to 4
Why AI needs GPUs	The maths inside a model, how a GPU works, and why memory speed sets the pace	5 to 8
From chip to cloud	Data centers, other AI chips, reading a spec sheet, and what it all means for your agent	9 to 12

Kai's rule: *everything in a data center, the power, the cooling and the networks, exists for one reason: to keep the GPUs busy doing useful work. Keep this in mind. It explains almost every design choice you will see.*

THERE ARE NO SILLY QUESTIONS

Q: Do I need to know hardware to run an agent?

A: Not to start. But when you choose a GPU, read a bill, or wonder why answers are slow, it helps a lot to know what is happening inside the machine. That is what this notebook is for.

Q: Is "the cloud" really just buildings full of computers?

A: Yes. When you rent a GPU from AWS, Google or Microsoft, it sits in a real server, in a real rack, in a real building, with real power and cooling. Chapter 9 walks through one.

Q: Do I need Notebook #2 first?

A: No. This notebook starts from zero. If you have read Notebook #2, many of its numbers, such as GPU memory sizes and tokens per second, will finally make sense.

Your turn

1. Put these in the order a request visits them: a GPU, a data center, the internet, a server, Priya's phone.

2. Priya is in Sri Lanka and the data center is in the United States. Which part of the trip is likely to take the longest: the trip inside the data center, or the trip across the world?

The answers are at the back.

Key takeaways

- A request travels from a phone, over the internet, into a data center, to a server and a GPU, and back.
- A round trip inside a data center takes well under a millisecond. Across the world, it takes a noticeable part of a second.
- A data center holds racks, racks hold servers, servers hold GPUs, and GPUs hold thousands of tiny workers.
- Everything in a data center exists to keep the GPUs busy.

Chapter two

2 Switches all the way down

Bits, bytes and transistors. Every number a computer knows is a row of tiny on/off switches.

Facts in this chapter last checked on 26 October 2026.



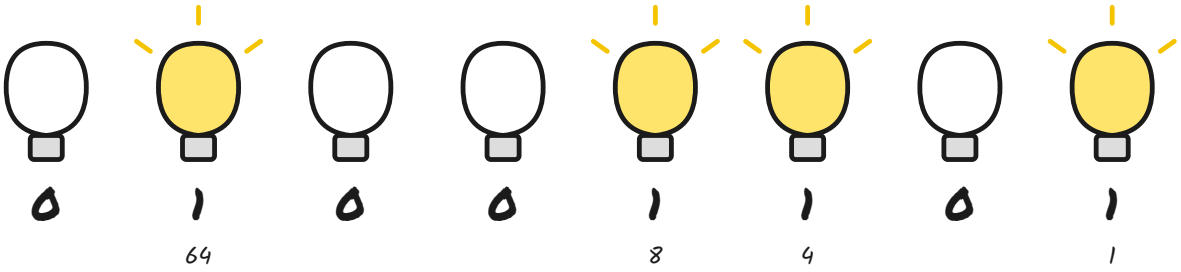
So everything, even Botley's model, is just switches?

Kai starts the tour at the smallest thing in the building. It is too small to see.

One switch, one bit

A switch can be **on** or **off**. Write on as **1** and off as **0**. One switch holds one **bit**: one 0 or 1.

One bit can't say much. But put 8 switches in a row and you get **256** different on/off patterns. Eight bits are called a **byte**. With a byte, you can store any whole number from 0 to 255.



64 + 8 + 4 + 1 = 77

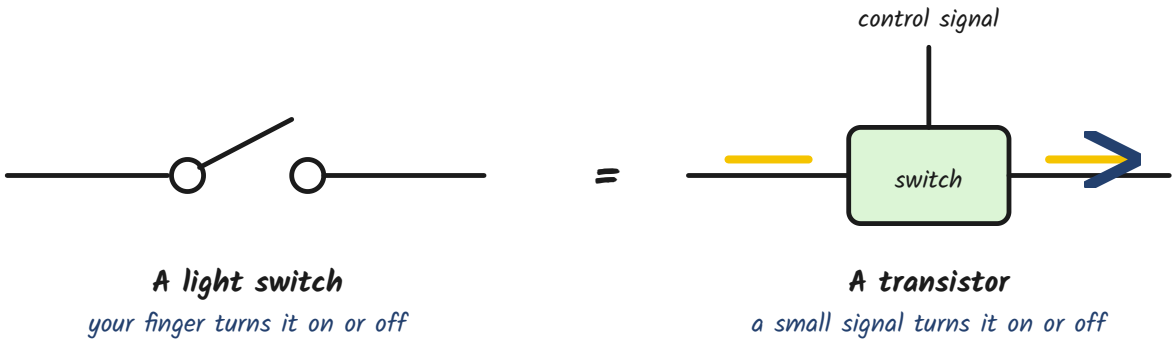
8 switches = 1 byte. 256 different on/off patterns.

Each switch has a value: 128, 64, 32, 16, 8, 4, 2, 1. Add up the ones that are on.

Everything in a computer is stored this way: letters, photos, your messages to Botley, and every number inside an AI model. It is all bytes. In the end, it is all switches.

The switch is a transistor

A light switch needs your finger. A computer's switch is a **transistor**: a tiny switch that a small electrical signal turns on or off. A transistor can switch billions of times a second, and because it is turned on and off by electricity, transistors can control other transistors. That is how a chip does logic and maths.

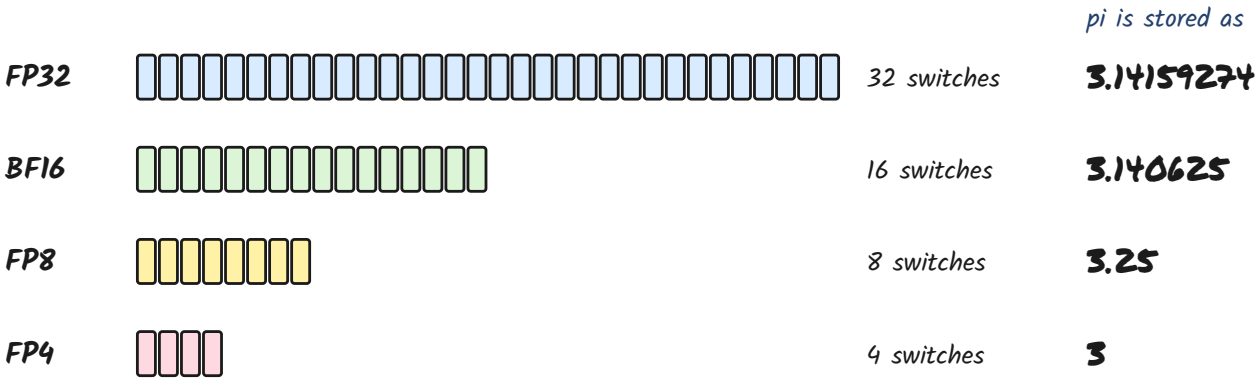


They are also incredibly small. NVIDIA's H100 GPU has **80 billion** transistors. Its Blackwell GPUs have **208 billion**, made by TSMC.

Sources: NVIDIA's Hopper architecture blog (80 billion transistors) and NVIDIA's Blackwell architecture page (208 billion transistors, TSMC 4NP process). Checked 26 October 2026.

Numbers with a decimal point

Whole numbers are easy. But a model's weights are numbers like 0.0213 or −1.402. Computers store these as **floating-point** numbers. Some of the bits say how big the number is, and the rest say its exact digits. The more bits you use, the more exact the number.



Fewer switches: less memory, faster to move, but a rougher number.

This is what the names in Notebook #2 mean:

Name	Bits per number	Bytes per number	How exact
FP32	32	4	Very exact
BF16	16	2	Exact enough for most AI models
FP8	8	1	Rougher
FP4	4	0.5	Very rough

So when someone says "this model is 8B in BF16, so 16 GB", they mean: 8 billion numbers × 2 bytes each. Fewer bits per number means less memory, less to move around, and faster maths, but rougher numbers.

Geek corner *(skip it if code isn't your thing)*

How the 32, 16, 8 and 4 bits are split. Each format has 1 sign bit (plus or minus), some **exponent** bits (how big) and some **mantissa** bits (the exact digits):

```
FP32      1 sign + 8 exponent + 23 mantissa
BF16      1 sign + 8 exponent +  7 mantissa
FP8 E4M3  1 sign + 4 exponent +  3 mantissa
FP4 E2M1  1 sign + 2 exponent +  1 mantissa
```

BF16 keeps the same 8 exponent bits as FP32, so it can hold numbers just as big or small. It only drops detail. That is why AI models like it so much.

Think about it

A GPU spends much of its time moving numbers from memory to its workers (Chapter 7). If each number takes half as many bits, what happens to how many numbers it can move each second?

There are no silly questions

Q: Why 8 bits in a byte, and not 10?

A: Mostly history. Early computers used different sizes, and 8 became the standard because it was a handy size for storing one letter. Today almost everything counts memory in bytes.

Q: If FP4 is so rough, why does anyone use it?

A: Because it is small and fast. New GPUs have special support for FP4 and FP8 maths. For many models, careful use of fewer bits loses very little quality. You still need to test it on your own tasks (Notebook #2, Chapter 4).

Your turn

1. Using the switch values 128, 64, 32, 16, 8, 4, 2, 1, which switches are on for the number 5?

2. A model has 70 billion weights stored in FP8. About how much memory do the weights need?

3. Why would moving FP8 numbers be faster than moving BF16 numbers?

The answers are at the back.

Key takeaways

- A bit is one on/off switch. A byte is 8 bits, enough for 256 patterns.
- Everything in a computer, including every weight in a model, is stored as bytes.
- A transistor is a tiny switch controlled by electricity. A modern GPU has tens to hundreds of billions of them.
- FP32, BF16, FP8 and FP4 use 32, 16, 8 and 4 bits per number. Fewer bits: less memory and faster, but rougher.



Chapter three

The CPU: a few very fast chefs

Instructions, the clock and cores. The part of the computer that runs Botley's own code.

Facts in this chapter last checked on 26 October 2026.

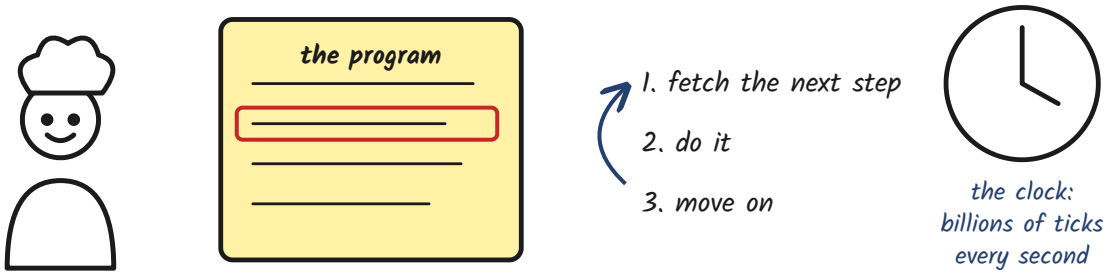


Before we meet the GPU, meet the chef that runs everything else.

Every computer, from a phone to a server, has a **CPU** (central processing unit). Billions of transistors are arranged so that the CPU can follow instructions.

A program is a recipe

A program is a long list of very small steps called **instructions**: "add these two numbers", "copy this number over there", "if this number is zero, jump to step 40". The CPU follows them one after another, like a chef following a recipe card.

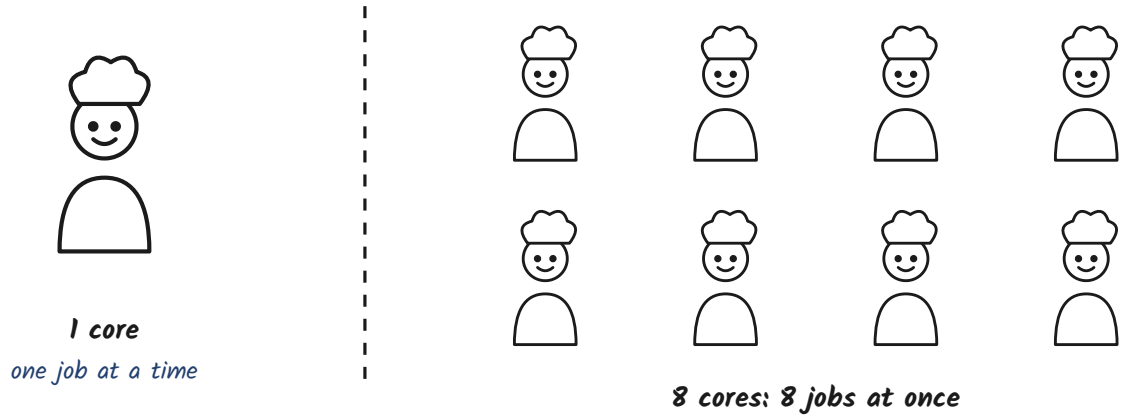


A CPU core repeats this loop billions of times a second.

The CPU repeats the same loop over and over: **fetch** the next instruction, **do** it, **move on**. A **clock** keeps time for this loop. A clock speed of 3 GHz means 3 billion ticks every second. Modern CPUs can often finish more than one instruction per tick.

More cores, more chefs

One core can only follow one list of instructions at a time. So modern CPUs have many **cores**, many chefs working side by side. Each core can run a different job at the same time.



A laptop CPU has a handful of cores. A big server CPU has many more:

Server CPU	Cores	Clock speed
AMD EPYC 9965	192	2.25 to 3.7 GHz
Intel Xeon 6980P	128	2.0 to 3.9 GHz

Sources: AMD EPYC 9965 and Intel Xeon 6980P product pages, checked 26 October 2026. Intel also makes Xeon 6 chips with up to 288 smaller, simpler cores.

What CPUs are great at

CPU cores are **clever**. Each one is very good at following complicated instructions full of decisions: "if the flight is full, try the next one; if the price is too high, ask the user". They switch quickly between many different jobs.

That is exactly what most of an agent's work looks like. The CPU runs:

- Botley's own code: planning steps and deciding what to do next
- the web server that receives Priya's message
- the tools: searching flights, checking the calendar, calling other services
- the database that stores conversations and memory
- turning text into tokens and back

> ⚠ **CAREFUL!**
>
> **The CPU is not the slow part of most agents.** The CPU work in an agent usually takes milliseconds. The model takes much longer. When an agent feels slow, look at the model calls and the tools first, not at the CPU.

What CPUs are not great at

Running a large AI model means doing billions of very simple multiplications, the same ones, over and over. A CPU can do it, but it has only tens or hundreds of chefs, and each chef is built for clever work, not for repeating one simple job billions of times. For that, you want a different kind of kitchen. That is Chapter 6.

There are no silly questions

Q: Can a CPU run an AI model at all?

A: Yes. Small models can run on a CPU, and tools such as llama.cpp are built for it. It works for testing or for very small models. For many users and bigger models, a GPU is much faster.

Q: Is a higher GHz always faster?

A: Not on its own. What matters is how much work gets done in total: the clock speed, how much each tick achieves, and how many cores share the job.

Your turn

1. Botley's code checks 20 flights one by one, with different rules for each airline. Is this a job for a CPU or a GPU?

2. A CPU runs at 2 GHz. How many clock ticks is that every second?

3. Name two parts of an agent system that run on a CPU.

The answers are at the back.

Key takeaways

- A CPU follows a program: a long list of small instructions, in a fetch, do, move-on loop.
- The clock sets the pace. 3 GHz means 3 billion ticks a second.
- Cores are chefs working side by side. Server CPUs have up to about 200 of them.
- CPUs are great at clever, varied work: an agent's code, tools, web servers and databases.
- They are not built for repeating one simple sum billions of times. That is the GPU's job.



Chapter four

The counter, the fridge and the warehouse

Memory and storage, from fastest to slowest, and why loading a model takes time.

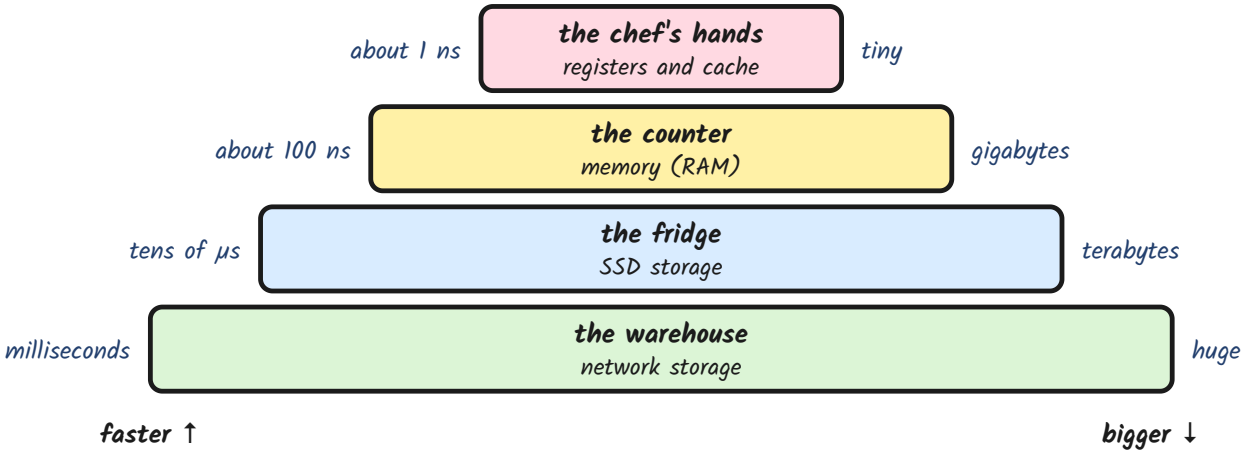
Facts in this chapter last checked on 26 October 2026.



Why does our model server take minutes to start? The GPU is so fast.

A chef can only cook with ingredients that are close at hand. Computers have the same problem. The places where data is kept come in a ladder: the closer to the worker, the faster but the smaller.

The storage ladder



In the kitchen	In the computer	Size	Time to reach
The chef's hands	**Registers** and **cache**, inside the chip	tiny	about a nanosecond
The counter	**Memory** (RAM)	gigabytes	about 100 nanoseconds
The fridge	**SSD** storage	terabytes	tens of microseconds, up to about 100
The warehouse across town	**Network storage**, such as Amazon S3 or Google Cloud Storage	as big as you like	milliseconds

The times come from Jeff Dean's "numbers everyone should know" (2009) and SSD makers' data sheets. They are rough sizes, not exact values, and newer hardware is faster. What matters is the gaps: each step down the ladder is many times slower than the one above it.

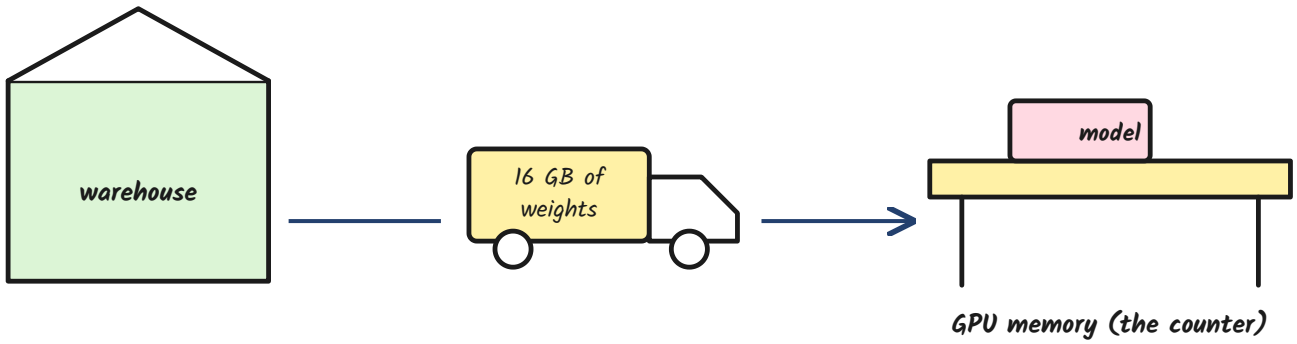
A nanosecond is a billionth of a second. A microsecond is a thousand nanoseconds, and a millisecond is a thousand microseconds. So reaching the warehouse can take a million times longer than reaching the chef's hands.

The GPU has its own counter

A GPU does not use the computer's normal memory for its work. It has its own memory built in, right next to its workers. This is the **GPU memory** from Notebook #2: 24 GB on an L4, 80 GB on an H100, and so on. Everything the GPU works on, the model's weights and every conversation's KV cache, has to be on this counter.

Between the computer's normal memory and the GPU's memory there is a connection called **PCIe**. A modern PCIe connection (Gen5, 16 lanes) moves about 64 GB per second in each direction. That sounds fast, but as Chapter 7 shows, the GPU's own memory is far faster.

Why loading a model takes time



Every time a model server starts, the weights travel from storage to the GPU first.

Before a model can answer anything, its weights must travel down the ladder: from the warehouse or the fridge, through the computer, over PCIe, onto the GPU's counter. For a model with 16 GB of weights, that is a lot of boxes to move. If the weights come from the internet or network storage, it can take minutes.

This is the slow start from Notebook #2, Chapter 7. Now you can see where the time goes, and why the fixes work: keep the weights closer (on a disk next to the GPU), or stream them in faster.

⚠ Careful!

Restarting a model server is never free. Each restart moves all the weights again. Plan updates so that at least one copy keeps running while another reloads.

Think about it

Maya's model server downloads 16 GB of weights from the internet every time it starts. Where on the ladder could she keep a copy to make the next start faster?

There are no silly questions

Q: Why not make all memory as fast as cache?

A: Fast memory is expensive, takes up space on the chip, and uses more power. So computers use a little very fast memory and a lot of slower memory, and try to keep the data they need right now in the fast part.

Q: Does the GPU need the computer's normal memory at all?

A: Yes, but less for the model itself. The computer's memory and CPU load the weights, run the serving software, and handle requests. The heavy maths happens in the GPU with its own memory.

Your turn

1. Put these in order from fastest to slowest: SSD, cache, network storage, RAM.

2. About how many nanoseconds are in one microsecond?

3. Where must a model's weights be before the GPU can use them?

The answers are at the back.

Key takeaways

- Storage comes in a ladder: cache, memory, SSD, network storage. Faster means smaller.
- Each step down the ladder is many times slower than the one above.
- The GPU has its own memory. The weights and KV cache must be there before it can work.
- Loading a model means moving all its weights down the ladder, which can take minutes.
- Keep weights close to the GPU to start faster.

Chapter five

5 The maths inside a model, gently

No equations. A model is mostly one tiny sum, repeated billions of times.

Facts in this chapter last checked on 26 October 2026.

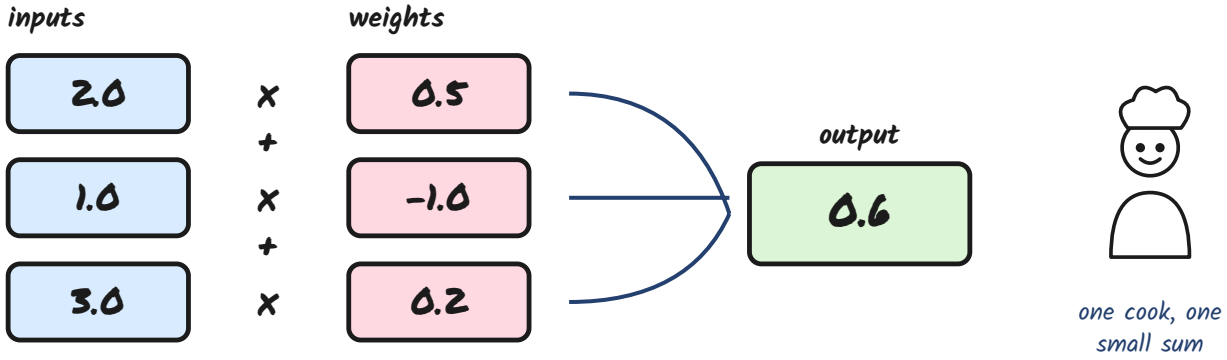


I always thought AI was something mysterious. Is it really just sums?

To understand why AI needs special chips, you need to know what kind of work a model does. The surprise is how simple each piece is.

One tiny sum

Most of the work in a model is one small sum, done over and over: take some numbers, multiply each one by a **weight**, and add up the results. This is called a **multiply-and-add**.



Multiply each input by its weight, then add them up. That's it.

That is it. The "intelligence" of a model is not in any single sum. It is in the billions of weights, which were set during training, and in doing these sums in the right order.

Many sums at once

A model is built in **layers**. Each layer takes the numbers from the layer before and does thousands of these sums to produce new numbers. Here is the important part: **each sum in a layer does not need the answer of any other sum in the same layer.** They can all be done at the same time.



Each sum doesn't need the others, so all of them can be done at the same time.

Work like this, where many pieces can be done side by side, is called **parallel** work. Some jobs are hard to split up, like following a recipe where step 2 needs the result of step 1. A model's layers are easy to split up.

How much work per token?

For every token it writes, a model does roughly one multiply-and-add for every weight it has.



Roughly one multiply-and-add for every weight in the model, for every single token.

So an 8B model does about 8 billion multiply-and-adds for each token. A 70B model does about 70 billion. And it does this again for the next token, and the next.

</> Geek corner *(skip it if code isn't your thing)*

Why "about"? The work per token is roughly one multiply-and-add per weight, which people often count as 2 operations (one multiply, one add). So an 8B model needs about 16 billion operations per token. Long conversations add some extra work for the attention part of the model, which looks back at earlier tokens using the KV cache. For mixture-of-experts models, only the active weights count.

Think about it

A CPU core does a handful of these sums per clock tick. Server CPUs have up to about 200 cores. Is that a good fit for billions of simple sums per token, for many users at once?

There are no silly questions

Q: Where does the "thinking" happen, then?

A: Nowhere in particular. Each sum is simple. What looks like thinking comes from billions of weights, tuned during training so that all those sums together produce a useful next token.

Q: Does a bigger model always do more work per token?

A: Yes, for a normal model: twice the weights means about twice the sums. Mixture-of-experts models are the exception. They only use some of their weights for each token (Notebook #2, Chapter 2).

Your turn

1. Work out the sum: inputs 1.0, 2.0 and 4.0; weights 0.5, 0.5 and 0.25.

2. Roughly how many multiply-and-adds does a 70B model do for one token?

3. Why can all the sums in one layer happen at the same time?

The answers are at the back.

Key takeaways

- Most of a model's work is multiply-and-add: multiply numbers by weights, then add them up.
- A model has layers, and the sums in a layer don't depend on each other, so they can run side by side.
- A model does about one multiply-and-add per weight for every token.
- That means billions of simple sums per token. The right tool is something with thousands of simple workers.

Chapter six

6 THE GPU: THOUSANDS OF LINE COOKS

What is inside a GPU, and why many simple workers beat a few clever ones for AI.

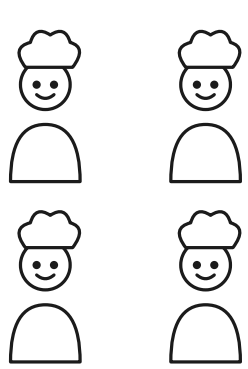
Facts in this chapter last checked on 26 October 2026.



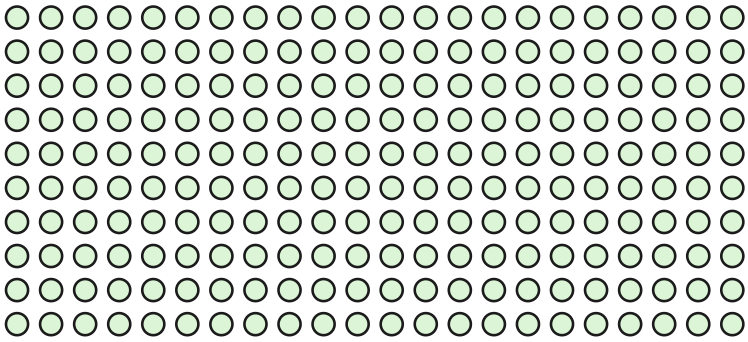
A CPU is a few top chefs. A GPU is a huge kitchen full of line cooks.

A **GPU** (graphics processing unit) was first built to draw video game graphics. Drawing a picture means working out the color of millions of pixels, and each pixel can be worked out on its own. That is parallel work, exactly like the sums in a model's layer. So the same chip turned out to be ideal for AI.

Few clever chefs, or thousands of simple cooks



CPU: a few expert chefs



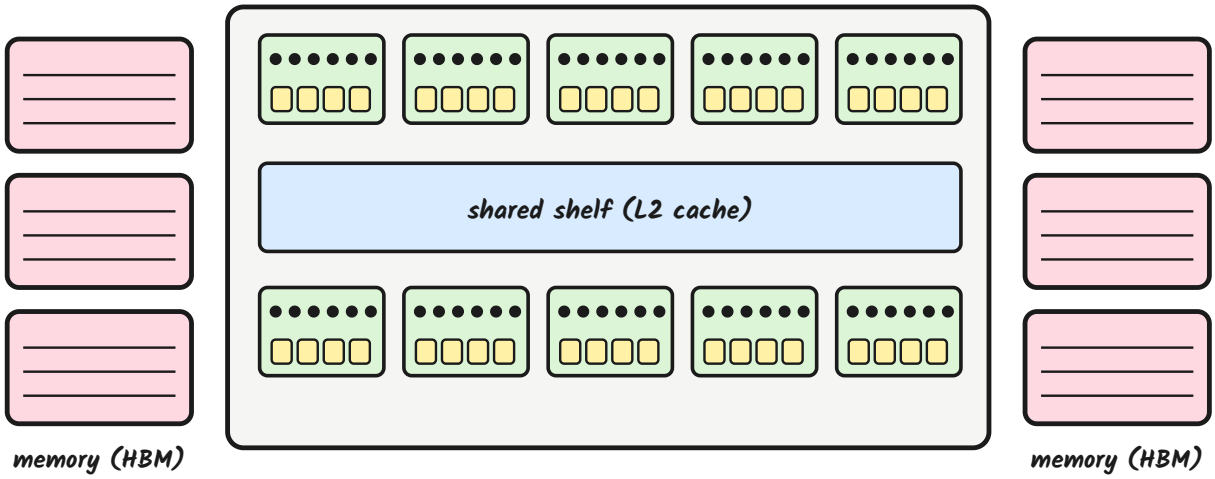
GPU: thousands of line cooks

A CPU core is clever: it is built to handle complicated instructions full of decisions, quickly. A GPU core is simple: it is built to do the same small job as thousands of other cores, at the same time.

	CPU	GPU (NVIDIA H100)
Workers	Up to about 200 clever cores	16,896 simple cores, plus 528 Tensor Cores
Good at	Varied work with lots of decisions	The same simple sum on lots of numbers at once
In an agent	Botley's code, tools, web server, database	The model

H100 numbers from NVIDIA's Hopper architecture blog: 132 streaming multiprocessors, each with 128 cores and 4 Tensor Cores. Checked 26 October 2026.

Inside a GPU



Each green station is a streaming multiprocessor (SM): cooks (dots) and maths machines (yellow).

A GPU is organised into stations called **streaming multiprocessors (SMs)**. NVIDIA describes a GPU as "a collection of Streaming Multiprocessors". Each SM has:

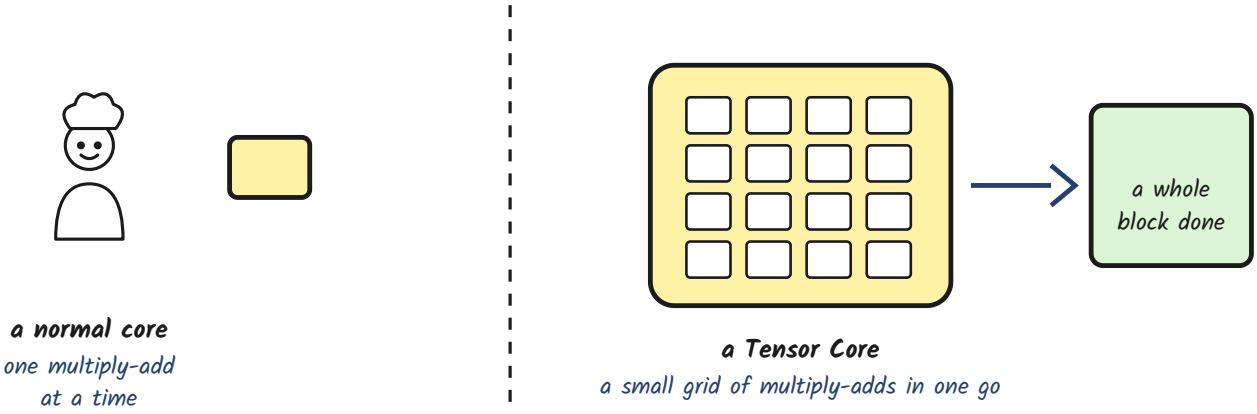
- many simple **cores** (the line cooks)
- a few **Tensor Cores** (special maths machines, see below)
- a small amount of very fast memory of its own

All the SMs share a bigger shelf, the **L2 cache**, and the GPU's main memory sits right next to the chip.

Inside an SM, cores work in teams of 32 called **warps**. All 32 do the same instruction at the same moment, each on its own piece of data. One instruction, 32 results.

Tensor Cores: a machine for grids of sums

A normal core does one multiply-and-add at a time. A **Tensor Core** does a whole small grid of them in one go. NVIDIA calls them "specialized high-performance compute cores for matrix multiply and accumulate". In plain words: they are built exactly for the multiply-and-add work that models are made of.



Most of a GPU's AI speed comes from its Tensor Cores. When a spec sheet says a GPU can do "989 teraFLOPS of BF16", that number is mostly Tensor Cores.

How fast is that?

A **FLOP** is one floating-point operation, such as one multiply or one add. Tera means a trillion.

GPU	BF16 maths, dense	FP8 maths, dense
L4	121 trillion operations per second	242 trillion
L40S	362 trillion	733 trillion
H100 SXM	about 989 trillion	about 1,979 trillion

Careful!

Watch for "with sparsity". NVIDIA's pages often show a bigger number "with sparsity", which is twice the normal (dense) number and only applies to specially prepared models. When you compare GPUs, use the dense numbers. The table above shows dense numbers.

An 8B model needs about 16 billion operations per token (Chapter 5). An H100 could, in theory, do that more than 60,000 times a second. But in practice, a single user gets far fewer tokens than that. The next chapter explains why.

There are no silly questions

Q: Why is it still called a graphics processor?
A: History. GPUs started in graphics, and the name stuck. Many data center GPUs, like the H100, have no video output at all.

Q: What is CUDA?
A: CUDA is NVIDIA's way of writing programs that run on its GPUs. Tools like PyTorch and vLLM use it for you, so you rarely write CUDA yourself.

Your turn

1. A job has 10 million independent pixel colors to work out. CPU or GPU?

2. What is the name for a team of 32 GPU cores doing the same instruction together?

3. A spec sheet shows "1,979 TFLOPS with sparsity". What is the dense number?

The answers are at the back.

Key takeaways

- A GPU has thousands of simple cores, built to do the same sum on lots of numbers at once.
- GPUs are organised into streaming multiprocessors (SMs). Cores work in teams of 32 called warps.
- Tensor Cores do whole grids of multiply-and-adds in one go. They do most of the AI work.
- Compare GPUs using dense numbers, not "with sparsity".
- In theory a GPU could write tens of thousands of tokens a second for one user. Something else holds it back.



Chapter seven

The real bottleneck

Getting ingredients to the cooks. Why memory speed, not maths speed, sets how fast your agent talks.

Facts in this chapter last checked on 26 October 2026.

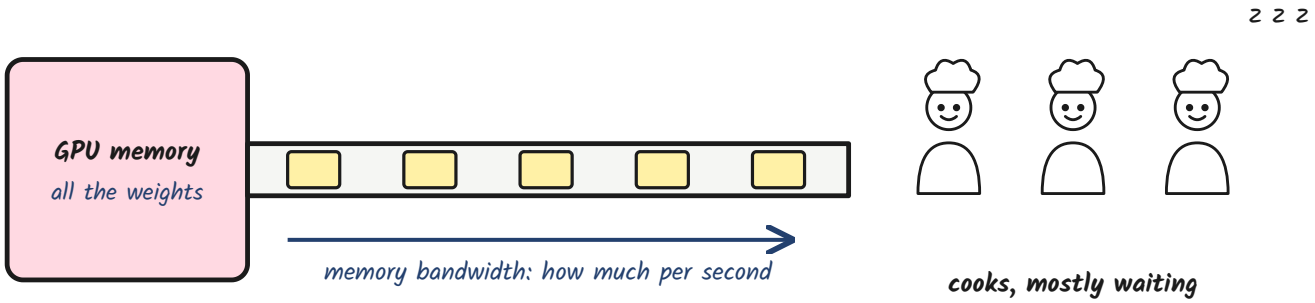


> *The cooks are never the problem. Getting food to them is.*

This is the most important chapter in the notebook. It explains why answers flow at the speed they do, why batching helps so much, and why GPU spec sheets brag about **memory bandwidth**.

Every token needs every weight

To write one token, the model does a multiply-and-add with **every** weight (Chapter 5). Every weight has to travel from the GPU's memory to the cores. Then, for the next token, all the weights have to travel again.



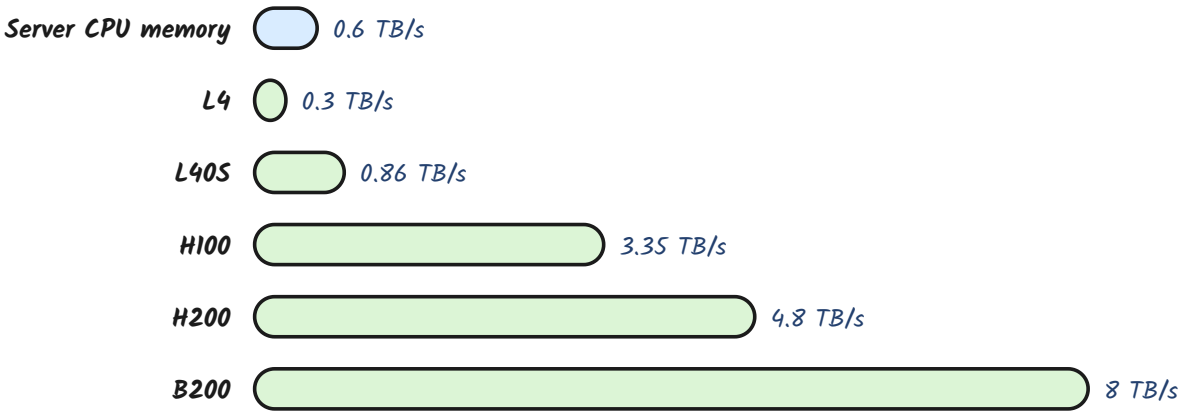
Writing one token needs every weight, so the belt has to deliver the whole model each time.

The PyTorch team put it this way: for one user, "each forward pass requires us to 'touch' every single parameter on the GPU", and "in order to generate 100 tokens, we must load our weights 100 times."

Source: PyTorch blog, "Accelerating Generative AI with PyTorch II: GPT, Fast". Databricks gives the same picture: a 7B model in 16-bit numbers has to move about 14 GB of weights for every token. Checked 26 October 2026.

Memory bandwidth: the width of the belt

Memory bandwidth is how many bytes per second can flow from memory to the cores. Think of it as the width of the conveyor belt.



Memory bandwidth: how fast numbers flow from memory to the workers.

Memory	Bandwidth
Server CPU (AMD EPYC 9965, 12 memory channels)	about 0.6 TB/s
L4	0.3 TB/s
L40S	0.86 TB/s
H100 SXM	3.35 TB/s
H200 SXM	4.8 TB/s
B200	about 8 TB/s

Sources: AMD EPYC 9965 product page, and NVIDIA product pages for the L4, L40S, H100, H200 and HGX B200. Checked 26 October 2026.

A rule of thumb you can use

For one user, the fastest a model can possibly write is roughly:

```
tokens per second  ≈  memory bandwidth  ÷  size of the weights
```

For an 8B model in BF16 (16 GB of weights):

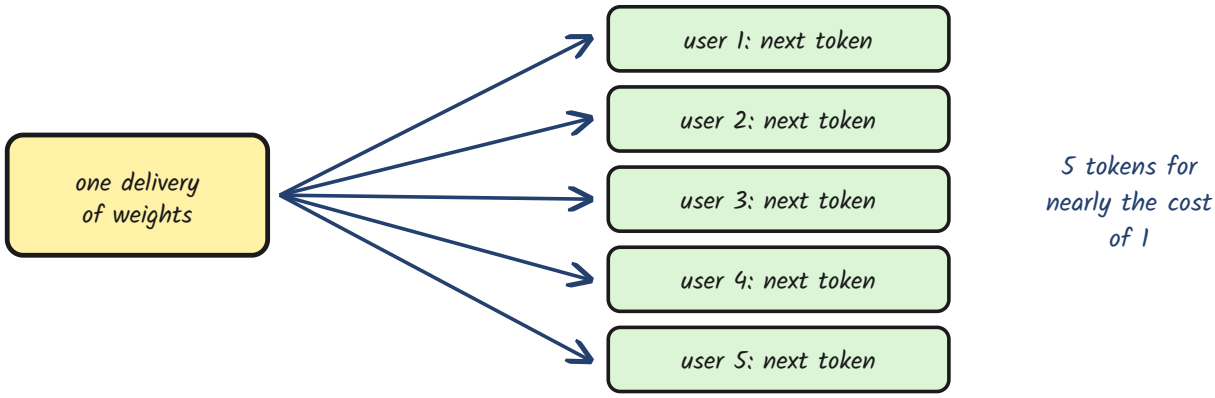
GPU	Bandwidth	Most tokens per second for one user
L4	0.3 TB/s	about 19
H100 SXM	3.35 TB/s	about 209
B200	8 TB/s	about 500

This is a ceiling, not a promise. Real numbers are lower, because the KV cache also has to be read and nothing runs perfectly. For example, NVIDIA measured about 71 tokens per second for one user of an 8B model in FP8 on an L40S. The rule gives a ceiling of 864 GB/s ÷ 8 GB, about 108. The real number sits below the ceiling, as it should.

> ⚠ **CAREFUL!**
>
> **Maths speed hardly matters for one user.** In Chapter 6, an H100 could in theory do the maths for over 60,000 tokens a second. The memory belt allows about 209. For a single user, the cooks spend most of their time waiting.

Batching: one delivery, many orders

If the weights have to travel anyway, why not use each delivery for more than one user? That is **batching**. The GPU brings in the weights once and writes the next token for many conversations at the same time.



This is why, in Notebook #2, one L40S wrote about 71 tokens a second for 1 user, but about 2,869 tokens a second in total for 100 users. The belt was the same. Each delivery just fed many more orders.

There is a limit. Each conversation needs its own KV cache, and that takes memory and bandwidth too. So the number of users per GPU depends on how much memory is left after the weights. This is why memory size and memory speed are the two numbers to look at first.

What makes a GPU fast for AI?

Maths speed	Memory speed
Look at all my teraFLOPS! I can do a thousand trillion sums a second.	
	And for one user, you sit waiting for me to bring the weights.
With batching, I get to use more of my power.	
	True. With many users, we both matter. But for answer speed, look at me first.

There are no silly questions

Q: Why not keep the weights inside the cores, where it's fast?

A: There's far too little room. The fast memory inside the GPU chip is measured in megabytes. A model's weights are measured in gigabytes.

Q: Does a smaller number format help with this?

A: Yes, a lot. FP8 weights are half the size of BF16 weights, so the belt can deliver them twice as fast. That is a big reason FP8 and FP4 are popular.

Q: Is the first token also limited by memory?

A: Less so. Reading the request (stage 1, Notebook #2, Chapter 3) handles all the request's tokens together, so each weight delivery is used many times. That stage is usually limited by maths speed instead.

Your turn

1. A 70B model in FP8 (70 GB) runs on one H200 (4.8 TB/s). Roughly what is the ceiling for one user's tokens per second?

2. Your GPU writes 30 tokens a second for one user. Will a GPU with twice the maths speed, but the same memory bandwidth, make one user's answers much faster?

3. Why does batching help so much?

The answers are at the back.

Key takeaways

- To write each token, every weight travels from memory to the cores.
- So for one user, memory bandwidth, not maths speed, sets how fast tokens appear.
- Rule of thumb: most tokens per second for one user ≈ bandwidth ÷ weight size. Real numbers are lower.
- Batching uses each delivery of weights for many users at once, so total throughput rises a lot.
- Smaller number formats (FP8, FP4) help because there are fewer bytes to move.



Chapter eight

When one GPU isn't enough

Linking GPUs together, splitting one model across them, and why GPU servers come with 8.

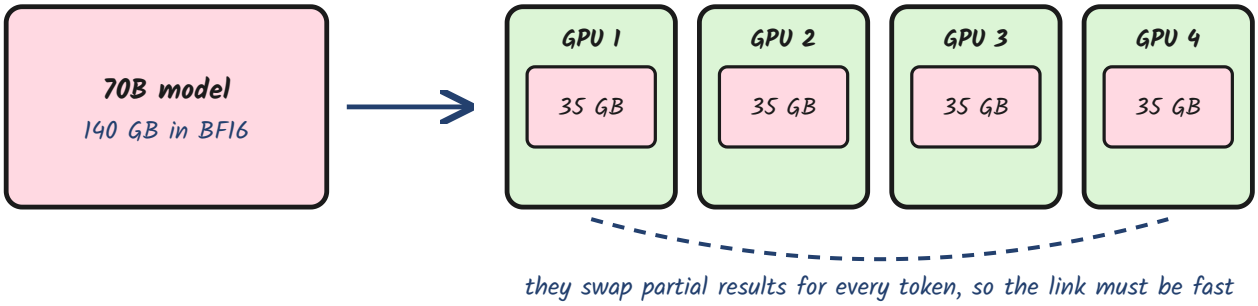
Facts in this chapter last checked on 26 October 2026.



Our big model doesn't fit on one GPU. Can't we just use two?

A 70B model in BF16 has about 140 GB of weights. An H100 has 80 GB of memory. It does not fit. The answer is to split the model across several GPUs. But splitting creates a new problem: the GPUs now have to talk to each other, a lot.

Splitting one model



The most common way is **tensor parallelism**, which Notebook #2 mentioned. Each GPU holds a slice of every layer. For each token, every GPU does the sums for its slice, then the GPUs swap their partial results before moving on to the next layer.

That swapping happens many times for every token. So the connection between the GPUs must be very fast, or the GPUs spend their time waiting for each other.

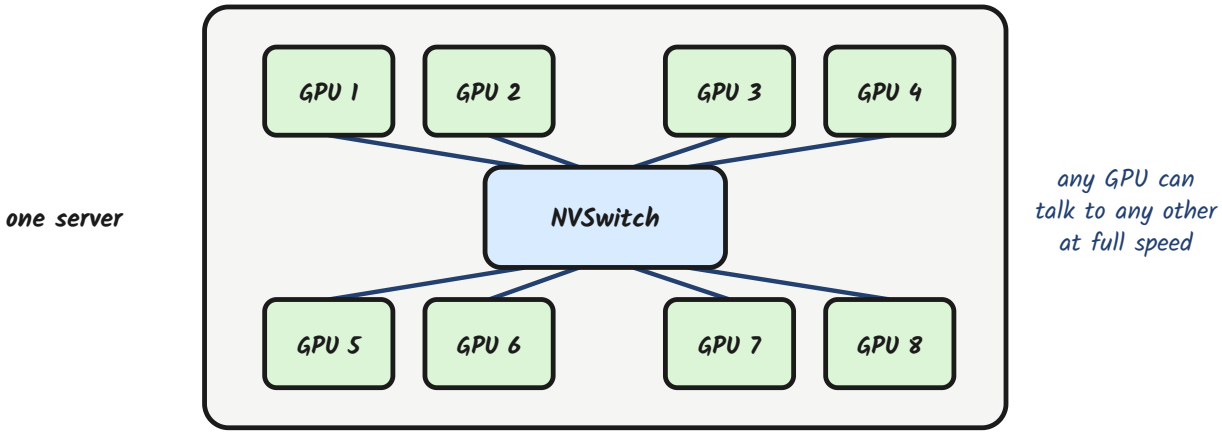
Two kinds of connection

Connection	What it links	Speed per GPU
PCIe Gen5 (16 lanes)	A GPU to the rest of the computer	about 128 GB/s (64 GB/s each way)
NVLink, H100	GPUs to each other	900 GB/s
NVLink, Blackwell	GPUs to each other	1.8 TB/s

Sources: NVIDIA's NVLink page and NVIDIA's Hopper architecture blog. Checked 26 October 2026.

NVLink is NVIDIA's own direct link between GPUs. On an H100 it is about 7 times faster than PCIe Gen5.

Why GPU servers come with 8



NVLink: about 900 GB/s per GPU on H100, 1.8 TB/s on Blackwell. PCIe Gen5: about 128 GB/s.

The big NVIDIA GPUs are usually sold on a board of **8 GPUs**, called **HGX**. The 8 GPUs connect through **NVSwitch** chips, so any GPU can talk to any other at full NVLink speed. Inside one server, the 8 GPUs can work together on one large model almost as if they were one big GPU.

This is why cloud machines like AWS p5.48xlarge, Google a3-highgpu-8g and Azure ND H100 v5 all have 8 GPUs.

Going bigger: a whole rack

NVIDIA's **GB200 NVL72** links **72** Blackwell GPUs and 36 Grace CPUs in one rack, all on NVLink. NVIDIA describes it as "a 72-GPU NVIDIA NVLink domain that acts as a single, massive GPU". Systems like this are used for the biggest models.

Between servers

When a job needs more GPUs than one NVLink group, servers talk over the data center network. That network is fast, but much slower than NVLink. Chapter 9 looks at it.

> ⚠ **Careful!**
>
> **Splitting is not free.** Two GPUs do not make a model twice as fast. Some time always goes to swapping results. Only split a model when it doesn't fit, or when you need more speed than one GPU can give. If a model fits on one GPU, one GPU per copy is usually the simplest and best choice.

> 💡 **Think about it**
>
> A 70B model in FP8 is about 70 GB. Would you split it across two H100s (80 GB each), or put it on one H200 (141 GB)? Think about the KV cache, and about the swapping.

There are no silly questions

Q: Can I split a model across GPUs that only have PCIe?

A: Yes, and some cloud machines do this. It works, but it is slower than NVLink. For splitting across many GPUs, NVLink matters a lot.

Q: Is this the same as running two copies of the model?

A: No. Running two copies (Notebook #2, Chapter 7) means each GPU has the whole model and serves different users. Splitting means several GPUs share one copy. You can do both: for example, 2 copies, each split across 4 GPUs.

Your turn

1. A model needs 300 GB for its weights. What is the smallest number of 80 GB GPUs that could hold it? Would that be enough in practice?

2. Roughly how many times faster is NVLink on an H100 than PCIe Gen5?

3. Why do GPU servers often have exactly 8 GPUs?

The answers are at the back.

Key takeaways

- When a model doesn't fit, split it across GPUs. The common way is tensor parallelism.
- Split GPUs swap results many times per token, so the link between them must be fast.
- NVLink connects GPUs directly: 900 GB/s on H100, 1.8 TB/s on Blackwell, versus about 128 GB/s for PCIe Gen5.
- GPU servers usually have 8 GPUs joined by NVSwitch. The GB200 NVL72 joins 72 in one rack.
- If a model fits on one GPU, keep it on one.

Chapter nine

9 Inside a data center

Racks, power, cooling and networks. Kai shows Maya around.

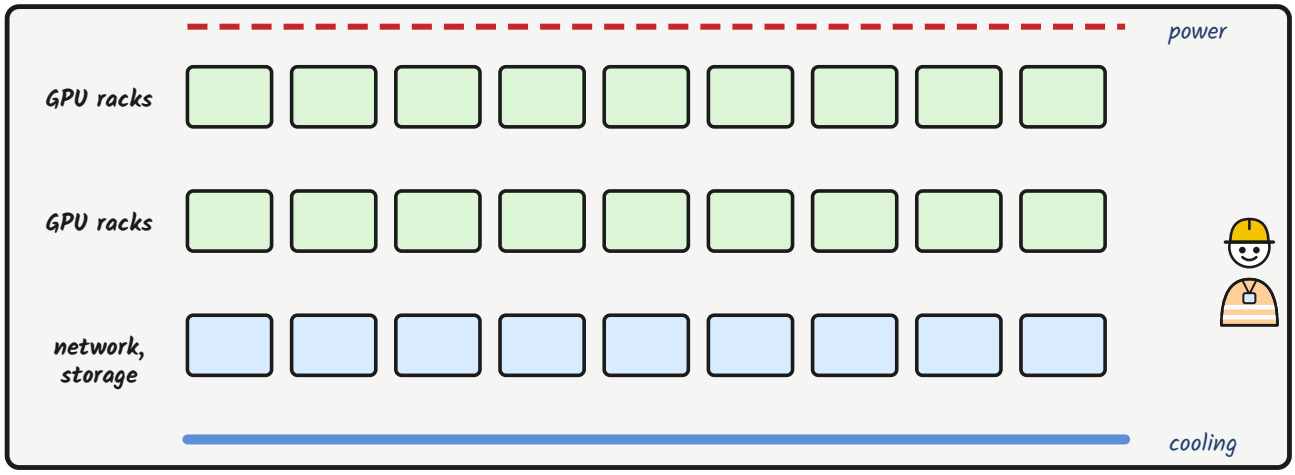
Facts in this chapter last checked on 26 October 2026.



Mind the cables. And yes, it's loud in here.

Kai badges Maya through two security doors into a huge, cool, noisy hall. Rows of tall cabinets stretch away in every direction, blinking with small lights.

Rows of racks

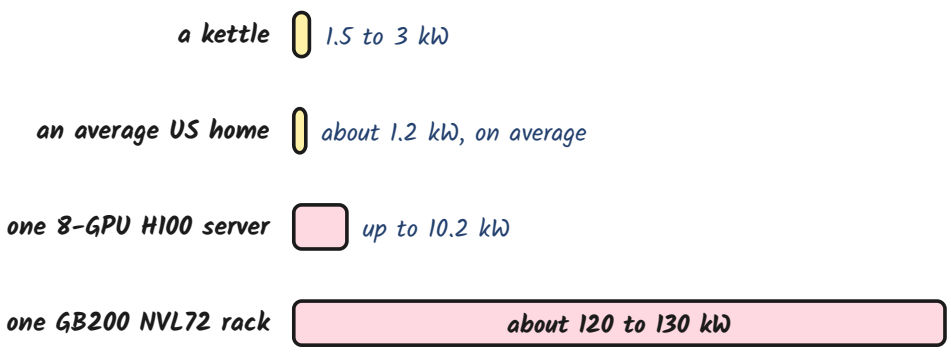


A **rack** is a tall metal cabinet that holds many servers stacked on top of each other. A data center hall holds rows and rows of racks. Some racks are full of GPU servers. Others hold network switches and storage.

Each rack needs three things from the building: **power** to run, **cooling** to remove the heat, and **network** cables to talk to everything else.

Power: the first limit

GPU servers use a lot of electricity, and almost all of it turns into heat.



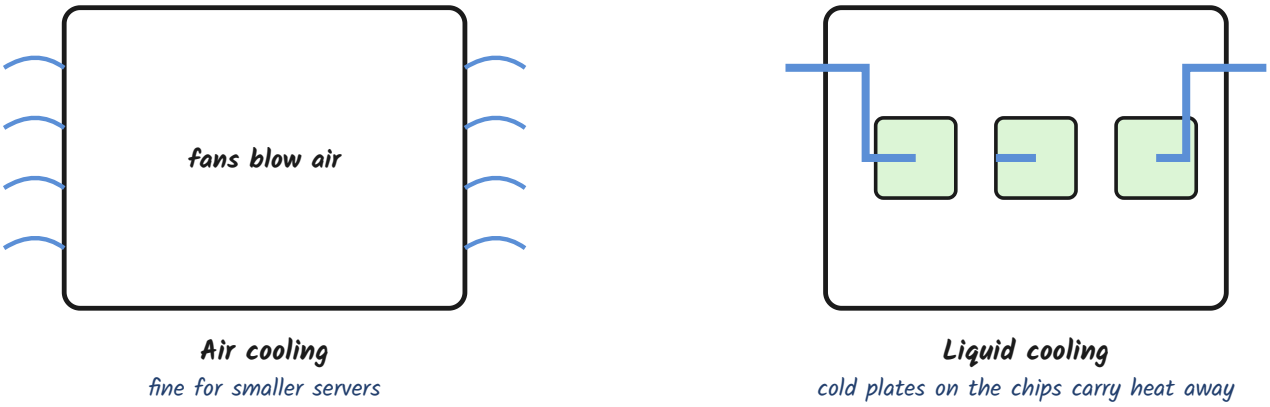
One big AI rack draws about as much power as 100 average US homes.

What	Power
An average rack in a data center today	about 5 to 15 kW
One NVIDIA DGX H100 server (8 GPUs)	up to 10.2 kW
One NVIDIA DGX B200 server (8 GPUs)	about 14.3 kW
One GB200 NVL72 rack (72 GPUs)	about 120 to 130 kW

Sources: NVIDIA's DGX H100 user guide (10.2 kW maximum) and DGX B200 page (about 14.3 kW). NVIDIA does not publish one figure for the GB200 NVL72 rack, so the range comes from server makers Supermicro (132 kW) and Gigabyte (up to 130 kW). The average-rack range is from Uptime Institute's 2026 survey. An average US home used about 865 kWh a month in 2024 (US EIA), which is about 1.2 kW on average. Checked 26 October 2026.

So a single big AI rack can draw as much power as about 100 average US homes. Many older data centers simply can't supply that much power to one rack, which is one reason new AI data centers are being built.

Cooling: getting rid of the heat



Most servers are cooled by **air**: fans push cool air through the servers, and the building's air conditioning takes the heat away. For the densest GPU racks, air is not enough. NVIDIA's Blackwell racks use **liquid cooling**: metal plates sit directly on the chips, and liquid flowing through them carries the heat away. NVIDIA says cooling can use up to 40% of a data center's electricity, and that liquid cooling can cut that a lot.

Networks: several of them

GPUs in different servers need to talk to each other, and to storage. NVIDIA's reference designs use several separate networks:

Network	What it carries
NVLink	GPU to GPU inside a server or rack (Chapter 8)
Compute network	GPU to GPU between servers, using InfiniBand or fast Ethernet
Storage network	Weights, data and checkpoints
Management networks	Logging in, setting up and watching the machines

Keeping these apart means a big data copy doesn't slow down the GPUs talking to each other.

Why GPUs are hard to get

Building all this takes time and a lot of money, and demand for GPUs is high. That's why the clouds have special ways to reserve them: **EC2 Capacity Blocks for ML** on AWS, which let you book GPUs for a set time in the future, **Flex-start**

on Google Cloud, and **capacity reservations** on Azure. Microsoft's documents point out that having **quota** (permission to use GPUs) is not the same as **capacity** (GPUs actually being free in that region).

Careful!

Plan for GPUs like you plan for a venue. For a launch, a big demo or a busy season, reserve GPUs ahead of time. Don't assume they will be free on the day.

There are no silly questions

Q: Why are data centers so loud?

A: Thousands of fans move huge amounts of air. People who work in the halls often wear ear protection.

Q: Do I need to know any of this to rent a GPU?

A: No. The cloud handles it. But it explains why GPUs are expensive, why some regions run out, and why cloud providers ask you to reserve big GPUs ahead of time.

Your turn

1. A data center can supply 15 kW to each rack. Can it run a GB200 NVL72 rack?

2. Why do the densest GPU racks use liquid cooling?

3. You have GPU quota in a region, but your GPU machine won't start. What could be the reason?

The answers are at the back.

Key takeaways

- A data center hall holds rows of racks. Each rack needs power, cooling and network.
- An 8-GPU server can draw over 10 kW. A GB200 NVL72 rack draws about 120 to 130 kW, as much as about 100 homes.
- The densest GPU racks need liquid cooling.
- Several separate networks connect GPUs, storage and management.
- GPUs are scarce. Quota is not the same as capacity, so reserve ahead for important events.

Chapter ten

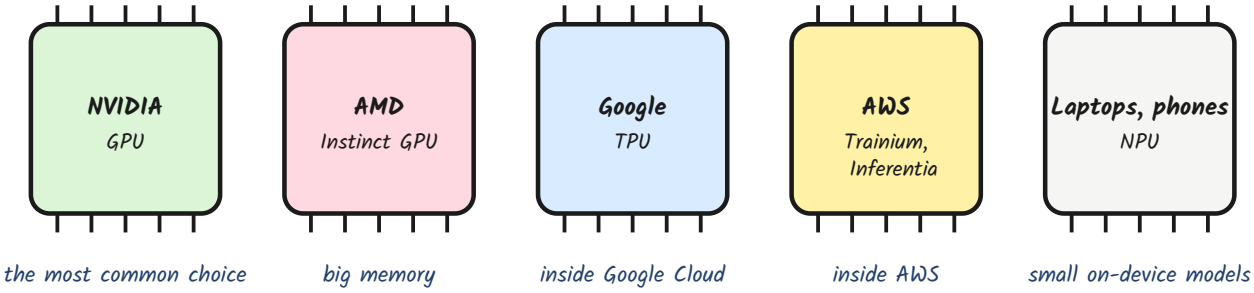
10 NOT ONLY NVIDIA

Google TPUs, AWS Trainium and Inferentia, AMD Instinct, and the AI chips inside laptops and phones.

Facts in this chapter last checked on 26 October 2026.



NVIDIA GPUs are the most common choice, but they are not the only AI chips. Each cloud has built its own, and AMD makes GPUs too. They all follow the same ideas from this notebook: lots of simple workers, fast memory right next to them, and fast links between chips.



The chips, side by side

Chip	Made by	Memory per chip	Memory bandwidth	Where you get it
TPU v5e	Google	16 GB	819 GB/s	Google Cloud
TPU v6e (Trillium)	Google	32 GB	about 1,600 GB/s	Google Cloud
TPU7x (Ironwood)	Google	192 GB	about 7.4 TB/s	Google Cloud
Inferentia2	AWS	32 GB	820 GB/s	AWS (inf2 machines)
Trainium2	AWS	96 GiB	2.9 TB/s	AWS (trn2 machines)
Trainium3	AWS	144 GB	4.9 TB/s	AWS (Trn3 UltraServers)
Instinct MI300X	AMD	192 GB	5.3 TB/s	Azure (ND MI300X v5), Oracle Cloud
Instinct MI355X	AMD	288 GB	8 TB/s	Oracle Cloud

Sources: Google Cloud TPU documents, AWS Neuron documents and the AWS Trn3 page, Oracle's MI355X announcement, and AMD and SemiAnalysis figures for the MI300X. Checked 26 October 2026. AMD's own product pages could not be reached during checking, so the MI300X figures come from SemiAnalysis.

Why choose something other than NVIDIA?

- **Price.** A cloud's own chips can cost less for the same work, especially on that cloud.
- **Availability.** When NVIDIA GPUs are hard to get, other chips may be free.
- **Memory.** AMD's Instinct chips have a lot of memory per chip, which helps big models fit.

The catch: software

Most AI software is built and tested first on NVIDIA GPUs, using CUDA. Other chips need their own software: AWS chips use the **Neuron** tools, Google TPUs use JAX, PyTorch with XLA, or vLLM's TPU support, and AMD uses **ROCm**. Many popular models and tools work on them, but not all, and not always on day one.

Careful!

Check your software first. Before you choose a non-NVIDIA chip, check that your model, your serving tool (for example vLLM) and the features you need are supported on it. Then measure speed and cost with your own requests.

AI chips in laptops and phones: NPUs

Many new laptops and phones include an **NPU** (neural processing unit), a small, low-power AI chip:

Device chip	NPU speed
Apple M4	38 trillion operations per second
Qualcomm Snapdragon X Elite	up to 45 trillion
Intel Core Ultra (Series 2, Lunar Lake)	up to 48 trillion
Microsoft's requirement for a Copilot+ PC	at least 40 trillion

NPUs are great for small models that run on the device: live captions, translating speech, blurring a video background, or a small assistant that works offline. They can't run large models like the ones behind most agents. They share the laptop's normal memory, which is much smaller and much slower than a data center GPU's memory (Chapter 7).

There are no silly questions

Q: Are TPUs and Trainium GPUs?

A: Not exactly. They are chips designed only for AI maths, without the graphics history. But they follow the same ideas: many simple workers, fast memory nearby and fast links between chips.

Q: Could Botley run on a laptop NPU?

A: Botley's code could, and a small model might. But a capable model for an agent like Botley needs far more memory and memory speed than an NPU has. Botley's model belongs in a data center.

Your turn

1. You need a chip with at least 150 GB of memory for one big model. Which chips in the table could hold it?

2. What should you check before moving from NVIDIA GPUs to another chip?

3. Why can't a laptop NPU run a large model?

The answers are at the back.

Key takeaways

- Google TPUs, AWS Trainium and Inferentia, and AMD Instinct GPUs are real alternatives to NVIDIA.
- They can be cheaper or easier to get, and AMD chips offer lots of memory.
- The catch is software: check that your model and serving tool support the chip.
- NPUs in laptops and phones run small on-device models, not the large models agents use.



Chapter eleven

How to read a GPU spec sheet

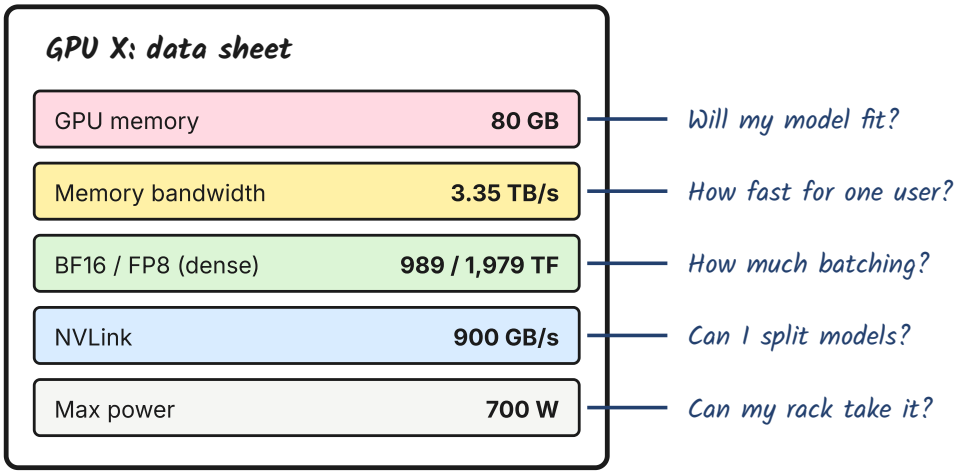
Five numbers, what each one tells you, and a worked example of choosing a GPU.

Facts in this chapter last checked on 26 October 2026.



Now the spec sheet finally makes sense. Let me pick a GPU.

A GPU spec sheet is full of numbers. For running a model, five of them matter most, and you now know what each one means.



The five numbers

Number	What it tells you	Chapter
GPU memory	Will the model and the KV cache fit?	4, and Notebook #2
Memory bandwidth	How fast can tokens appear for each user?	7
Maths speed (dense, at the number format you use)	How much batching can it handle, and how fast is the first token?	6
NVLink	Can you split a big model across GPUs well?	8
Power	Can your rack or data center supply it?	9

The line-up

GPU	Memory	Bandwidth	BF16 dense	FP8 dense	Power
L4	24 GB	300 GB/s	121 TF	242 TF	72 W
L40S	48 GB	864 GB/s	362 TF	733 TF	350 W
A100 80GB (SXM)	80 GB	2,039 GB/s	312 TF	not supported	400 W
H100 (SXM)	80 GB	3,350 GB/s	about 989 TF	about 1,979 TF	up to 700 W
H200 (SXM)	141 GB	4,800 GB/s	about 989 TF	about 1,979 TF	up to 700 W

GPU	Memory	Bandwidth	BF16 dense	FP8 dense	Power
B200 (in an 8-GPU HGX board)	180 GB	about 8,000 GB/s	about 2,250 TF	about 4,500 TF	up to 1,200 W

Sources: NVIDIA's product pages for each GPU, NVIDIA's HGX page (B200 figures are the 8-GPU totals divided by 8), and NVIDIA's Blackwell Ultra blog (B200 maximum power). TF means trillion operations per second. NVIDIA's pages often quote numbers "with sparsity", which are twice these. Checked 26 October 2026.

Look at the H100 and the H200. The maths speed is the same. But the H200 has more memory and about 43% more memory bandwidth. For serving models, that makes the H200 noticeably faster per user, and it can hold bigger models or more conversations.

A worked example

Maya wants to serve an 8B model in FP8 (8 GB of weights) for about 50 users at once. Which GPU?

1. **Memory.** 8 GB of weights. The L4's 24 GB fits it, with room for some KV cache. The L40S's 48 GB leaves much more room for 50 conversations.
2. **Bandwidth.** The ceiling for one user is about 300 ÷ 8 ≈ 37 tokens per second on the L4, and 864 ÷ 8 ≈ 108 on the L40S. With 50 users sharing, each gets less. The L40S leaves much more headroom.
3. **Price.** On AWS, an L4 machine (g6.xlarge) is about $0.80 an hour and an L40S machine (g6e.xlarge) about $1.86 an hour (Notebook #2, Chapter 4).
4. **Decision.** Maya starts with the L40S, then load-tests it with Botley's real requests before deciding how many she needs.

⚠ Careful!

The spec sheet is the starting point, not the answer. Real speed depends on your model, your serving software, your request lengths and how many users you have. Always measure with your own traffic.

There are no silly questions

Q: Why does the A100 say FP8 is not supported?

A: The A100 is an older design. It has no special hardware for FP8 maths. It can still store FP8 weights in some software, but it doesn't get the FP8 speed-up.

Q: Which single number should I look at first?

A: Memory size, to see if the model fits. Then memory bandwidth, to see how fast it will feel.

Your turn

1. A 70B model in FP8 (70 GB) must run on one GPU with room for the KV cache. Which GPUs in the table could work?

2. Using the rule from Chapter 7, roughly how many tokens per second could one user get from that model on an H200?

3. Two GPUs have the same maths speed, but one has 40% more memory bandwidth. Which is better for serving one user quickly?

The answers are at the back.

Key takeaways

- Five numbers matter most: memory, memory bandwidth, dense maths speed, NVLink and power.
- Memory decides what fits. Bandwidth decides how fast it feels. Maths speed decides how much batching you can do.
- The H200 has the same maths speed as the H100, but more and faster memory, so it serves models better.
- Use the spec sheet to shortlist, then measure with your own requests.

Chapter twelve

12 What this means for your agent

Where the seconds go, where the money goes, and when a plain CPU is enough.

Facts in this chapter last checked on 26 October 2026.

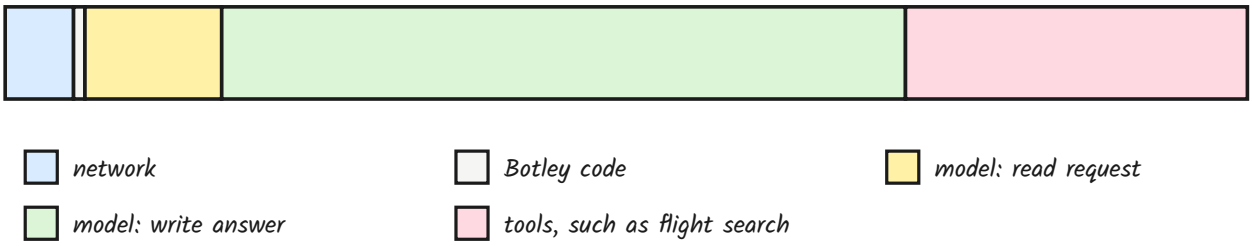


So that's what happens every time I think. Thanks, Kai!

The tour is over. Maya and Kai sit down with a coffee and turn everything Maya saw into practical lessons for Botley.

Where the seconds go

Where the seconds go in one Botley task (an example, not a measurement)



Writing the answer is usually the biggest part, and memory speed sets its pace.

When one Botley task feels slow, the time is spread across several places. The sizes above are only an example. Measure your own agent to see the real split. But in most agents, the model writing its answers is the biggest piece, followed by the tools.

If this is slow	Why	What helps
The wait before the first word	Reading a long request (Notebook #2, Chapter 3)	Shorter instructions, fewer tool results, prompt caching
The words flowing out	Memory bandwidth (Chapter 7)	A GPU with more bandwidth, a smaller or FP8 model, shorter answers
The tools	The services Botley calls	Faster tools, calling tools in parallel
The trip across the world	Distance (Chapter 1)	Running the model in a region near your users

Where the money goes

You pay for GPUs by the hour, whether they are busy or not. So the two biggest cost levers are:

1. **Keep GPUs busy.** Batching many users onto one GPU (Chapter 7) makes each token far cheaper.
2. **Use the smallest model and number format that does the job well.** Fewer bytes to move means fewer GPUs for the same traffic.

Notebook #2, Chapter 9 has the full cost math.

When a plain CPU is enough

Not everything in an agent needs a GPU. Botley's own code, the web server, the database and the tools all run happily on CPUs (Chapter 3). And if you use a managed model service, someone else runs the GPUs and you don't need any of your own.

You need GPUs of your own only when you run a model yourself. Notebook #2 helps you decide whether that's worth it.

What Maya learned

Maya's notes from the tour

- Every token needs every weight, so **memory bandwidth** sets the pace for each user.
- **Batching** shares each delivery of weights between users. That's why one GPU can serve many people.
- **Memory size** decides what fits: the model plus every conversation's KV cache.
- **FP8 and FP4** help because there are fewer bytes to move.
- Big models are **split across GPUs**, which need fast links like NVLink.
- GPUs live in **real buildings** with power and cooling limits. That's why they're scarce and expensive.

A checklist for the hardware side

- [ ] I know how big my model's weights are in the number format I use (Chapter 2)
- [ ] I know the GPU memory, memory bandwidth and dense maths speed of my GPU (Chapter 11)
- [ ] I have worked out the one-user ceiling: bandwidth ÷ weight size (Chapter 7)
- [ ] I have load-tested with real requests and many users at once
- [ ] My model runs in a region close to my users (Chapter 1)
- [ ] I have reserved GPUs ahead for big events (Chapter 9)

There are no silly questions

Q: Will GPUs keep getting faster?

A: Yes. Each new generation has added more memory, more bandwidth and faster maths. But the ideas in this notebook stay the same, so you will be able to read the next spec sheet too.

Q: What should I read next?

A: Notebook #2, Running Your Own Models, puts this into practice: choosing between managed services and your own GPUs, serving a model with vLLM, and doing the cost math.

Your turn

1. Botley's answers start quickly but then flow slowly. Name two things that could help.

2. Your agent uses a managed model service. Do you need GPUs of your own?

3. Name the two biggest levers for lowering the cost of running your own model.

The answers are at the back.

Key takeaways

- In most agents, the model writing its answer takes the most time, then the tools.
- Slow first words: shorten the request. Slow flowing words: more memory bandwidth, or fewer bytes per weight.
- Keep GPUs busy with batching, and use the smallest model that does the job well.
- Agent code, tools and databases run on CPUs. You need your own GPUs only if you run the model yourself.

Word list and answers

Every tricky word in this notebook, in plain English, and the answers to the exercises

Word	What it means
Bit	One on/off switch, written as 1 or 0.
Byte	8 bits. Enough for 256 different patterns.
Transistor	A tiny switch that electricity turns on or off. Chips are made of billions of them.
Floating-point number	A number with a decimal point, stored as a size part and a digits part.
FP32, BF16, FP8, FP4	Number formats using 32, 16, 8 and 4 bits. Fewer bits: less memory, rougher numbers.
CPU	The chip that runs a computer's programs. A few clever cores.
Core	One worker inside a chip that follows instructions.
Clock speed, GHz	How many ticks per second a chip's clock makes. 3 GHz is 3 billion ticks a second.
Instruction	One tiny step in a program, such as "add these two numbers".
Cache	Very small, very fast memory inside a chip.
Memory, RAM	Where a computer keeps what it is working on now. Fast, but cleared when the power is off.
SSD	Fast storage that keeps data when the power is off.
PCIe	The connection between a GPU and the rest of the computer.
Multiply-and-add	Multiply numbers by weights and add them up. Most of a model's work.
Layer	One step of a model. The sums inside a layer don't depend on each other.
Parallel work	Work that can be split into pieces done at the same time.
GPU	A chip with thousands of simple cores, built for parallel work.
Streaming multiprocessor (SM)	One station inside a GPU, with its own cores, Tensor Cores and small fast memory.
Warp	A team of 32 GPU cores doing the same instruction together.
Tensor Core	A part of a GPU that does a whole grid of multiply-and-adds in one go.
FLOPS, TFLOPS	Floating-point operations per second. Tera means a trillion.
Dense, with sparsity	Dense is the normal speed. "With sparsity" is twice that, and only for specially prepared models.
HBM	High bandwidth memory: memory stacked right next to the GPU chip, very fast.
GDDR	Graphics memory on separate chips around the GPU. Cheaper, slower than HBM.
Memory bandwidth	How many bytes per second can flow from memory to the cores.
Batching	Using each delivery of weights to write tokens for many users at once.
Tensor parallelism	Splitting one model across several GPUs, each holding a slice of every layer.
NVLink	NVIDIA's fast direct link between GPUs.
NVSwitch	A chip that lets every GPU in a group talk to every other at full NVLink speed.

Word	What it means
HGX	NVIDIA's board of 8 GPUs joined by NVSwitch.
Rack	A tall cabinet holding many servers.
kW (kilowatt)	A measure of power. A kettle uses about 1.5 to 3 kW.
Liquid cooling	Cooling chips with liquid flowing through metal plates on top of them.
Quota, capacity	Quota is permission to use GPUs. Capacity is GPUs actually being free.
TPU, Trainium, Inferentia	AI chips made by Google (TPU) and AWS (Trainium, Inferentia).
NPU	A small, low-power AI chip inside laptops and phones.

Answers

Chapter 1, Your turn

1. Priya's phone, the internet, a data center, a server, a GPU.
2. The trip across the world. Inside a data center, a round trip takes well under a millisecond. Across the world, it takes a noticeable part of a second.

Chapter 2, Your turn

1. The switches worth 4 and 1: 00000101.
2. About **70 GB** (70 billion × 1 byte).
3. Each FP8 number is half the size, so twice as many numbers can be moved every second.

Chapter 3, Your turn

1. A CPU. It is varied work full of decisions.
2. 2 billion ticks.
3. For example: Botley's own code, the web server, the tools, the database, or turning text into tokens.

Chapter 4, Your turn

1. Cache, RAM, SSD, network storage.
2. 1,000.
3. In the GPU's own memory.

Chapter 5, Your turn

1. 1.0 × 0.5 + 2.0 × 0.5 + 4.0 × 0.25 = 0.5 + 1.0 + 1.0 = **2.5**.
2. About **70 billion**.
3. Each sum only needs the inputs and its own weights, not the answer of any other sum.

Chapter 6, Your turn

1. A GPU. The colors can all be worked out at the same time.
2. A warp.
3. About **989** TFLOPS (half of 1,979).

Chapter 7, Your turn

1. About **69** tokens per second (4,800 GB/s ÷ 70 GB). Real numbers will be lower.
2. No, not much. For one user, memory bandwidth sets the pace, not maths speed.
3. The GPU brings in the weights once and uses them to write the next token for many users at the same time.

Chapter 8, Your turn

1. 4 GPUs hold 320 GB, but that leaves only about 20 GB in total for the KV cache and working space, which is too little in practice. Use more GPUs (for example a full 8-GPU server), GPUs with more memory, or FP8 to shrink the weights.
2. About 7 times (900 GB/s ÷ 128 GB/s).
3. 8 GPUs joined by NVSwitch can all talk to each other at full NVLink speed, so they can share one large model well. NVIDIA sells them as an 8-GPU HGX board.

Chapter 9, Your turn

1. No. A GB200 NVL72 rack needs about 120 to 130 kW.
2. They make so much heat in so little space that air can't carry it away fast enough.
3. Quota is permission, not a guarantee. There may be no free GPUs of that kind in that region or zone right now. Try another zone or region, or reserve capacity ahead.

Chapter 10, Your turn

1. TPU7x (192 GB), Instinct MI300X (192 GB) and Instinct MI355X (288 GB).
2. That your model, your serving tool and the features you need are supported on that chip. Then measure speed and cost.
3. It shares the laptop's normal memory, which is too small to hold a large model and much too slow to feed it quickly.

Chapter 11, Your turn

1. The H200 (141 GB) and the B200 (180 GB). The H100's 80 GB would leave almost no room for the KV cache.
2. About **69** tokens per second (4,800 ÷ 70). Real numbers will be lower.
3. The one with more memory bandwidth.

Chapter 12, Your turn

1. For example: a GPU with more memory bandwidth, a smaller or FP8 model, or shorter answers.
2. No. The service runs the GPUs. Botley's own code can run on CPUs.
3. Keep GPUs busy with batching, and use the smallest model and number format that does the job well.